



Distributed Computing (CCZG526/SSZG526)

ASSIGNMENT II

Submission Details, Code Repository and Video Link

Group Number	33
Language Used	Python 3
Algorithm Used	Ricart–Agrawala Distributed Mutual Exclusion (1981)
Code location (also included in document)	https://github.com/gchinar/Gp33-distributed-computing-assignment2
Video/Snapshots location	https://www.youtube.com/watch?v=cgv9Sd_cZY4 https://github.com/gchinar/Gp33-distributed-computing-assignment2/tree/main/demo/

Table of Contents

Submission Details, Code Repository and Video Link	1
1. Contribution Table	3
2. Test Cases	4
a. TC1 - Single client posts (no contention)	5
b. TC2 - Concurrent post by both clients	6
c. TC3 - view bypasses the DME protocol	8
d. TC4 - Multiple sequential posts	8
e. TC5 - Stress test: 20 interleaved concurrent posts	9
f. TC6 - DME log inspection	10
3. Additional features added other than mentioned in the problem statement	11
4. Distributed Application Code	11
server.py - File-server module	11
client.py - Chat-application module	15
Configuration files	22
5. Distributed Mutual Exclusion Code	24
6. Other Supporting Sections	35
6.1 Problem Statement and Approach	35
6.2 Architecture	35
6.3 Ricart-Agrawala Algorithm Summary	36
7. Snapshots	38
8. Requirement Coverage Matrix	46
9. Cloud-Lab Deployment Commands	48

1. Contribution Table:

Sl. No.	Name	ID No.	Contribution
1	Amit Kumar Jha	2025mt03001	25% Created & deployed server.py (file server) Ran SERVER node during the demo
2	Chinar Goel	2025mt03125	25% Created & deployed client.py (chat application) Demo coordination & Assignment documentation
3	Jyoti	2025mt03068	25% Integration testing across 3 cloud nodes Ran CLIENT B node during the demo
4	Satya Prakash Gupta	2025mt03128	25% Created & deployed dme.py (Ricart–Agrawala module) Ran CLIENT A node during the demo

2. Test Cases :

Screenshots: <https://github.com/gchinar/Gp33-distributed-computing-assignment2/tree/main/demo/snapshots>

Test ID	Scenario	Expected Result	Actual Result	Status
TC1	Single client posts (no contention)	Client acquires CS without any deferral and post is committed on the file server.	Client A (Satya: post crash) acquired CS immediately after one REPLY from B; CS held for 0.009s; chat.txt updated.	PASS
TC2	Concurrent post by both clients	Lamport timestamp tie-break decides the winner; the loser defers and is granted afterwards. No CS overlap.	Live demo shows Client A (Satya: test344, ts=1) entering CS first and Client B (Jyoti: Hello Satya, ts=3) waiting for A's REPLY; additional simultaneous-request evidence shows the (1,'A') vs (1,'B') tie-break with DEFER and deferred REPLY.	PASS
TC3	view during/after post on the other client	view bypasses the DME protocol and returns immediately.	Server log shows view() RPCs interleaved freely with post() RPCs, served without any DME exchange.	PASS
TC4	Multiple sequential posts on each client	Each post performs a fresh REQUEST/REPLY round-trip; chat.txt contains all messages in order.	Both clients posted multiple messages across the demo session; chat.txt shows all 13 messages in chronological order.	PASS

Test ID	Scenario	Expected Result	Actual Result	Status
TC5	Stress: 20 interleaved concurrent posts (10 per client)	All 20 messages reach the file; no critical-section overlap is detected.	All 20 messages present in chat.txt; instrumentation detected zero overlaps; messages alternate cleanly between A and B.	PASS
TC6	DME log inspection	Logs show REQUEST, REPLY, DEFER/REPLY-decision, ENTER CS and EXIT CS events with Lamport timestamps.	All algorithm transitions are visible in TC1/TC2 screenshots and in the added simultaneous-request DEFER evidence; logs are consistent with Ricart–Agrawala.	PASS

a. TC1 – Single client posts (no contention)

Expected Result: When only one client is active, it should obtain the CS as soon as the single peer responds to its REQUEST. No DEFER occurs because the peer is in RELEASED state.

Command used

```
# On Client A (Satya's node)
Satya_machine> post crash
```

Actual terminal output (Client A logs)

```

Satya_machine> post crash
16:28:22 [A] -> requesting CS for post
16:28:22 [A] REQUEST CS ts=(11, 'A') -> broadcasting to ['B']
16:28:22 [A] RECV REPLY from B (1/1 collected)
16:28:22 [A] >>> ENTER CS ts=(11, 'A') (got 1 REPLYs)
16:28:22 [A] in CS, calling server.post()
16:28:22 [A] post committed on server
16:28:22 [A] <<< EXIT CS -> sending deferred REPLYs to []
16:28:22 [A] CS held for 0.009s
Satya_machine> view
25 Apr 4:24PM Satya: satya
25 Apr 4:24PM Satya: helo jyoti
25 Apr 4:25PM Satya: amit
25 Apr 4:26PM Jyoti: hello
25 Apr 4:27PM Jyoti: hello everyone
25 Apr 4:28PM Satya: crash
Satya_machine>

```

Figure 5.1 — Client A: full Ricart–Agrawala cycle for a single post (no contention). *REQUEST* → *RECV REPLY* → *ENTER CS* → *post committed* → *EXIT CS*. CS held for 9 ms.

The five algorithmic events all appear in order: requesting CS, broadcasting REQUEST with timestamp (11, 'A'), collecting REPLY from B, entering CS, calling server.post(), and finally exiting CS. The subsequent view command shows the new message has been appended to the shared file.

b. TC2 – Concurrent post by both clients

Expected Result: Both clients move into WANTED with similar Lamport timestamps. The (clock, id) tie-break decides who enters first; the loser defers its REPLY until the winner releases. No CS overlap.

Commands used (run nearly simultaneously)

```

# On Client A (Satya)

Satya_machine> post test344

# On Client B (Jyoti)

Jyoti_machine> post Hello Satya

```

Client A side — receives B's REQUEST while RELEASED, replies immediately

```

10:57:00 [A] >>> ENTER CS  ts=(1, 'A')  (got 1 REPLYs)
10:57:00 [A] in CS, calling server.post()
10:57:00 [A] post committed on server
10:57:00 [A] <<< EXIT CS  -> sending deferred REPLYs to []
10:57:00 [A] CS held for 0.010s
Satya_machine> view
25 Apr 4:24PM Satya: satya
25 Apr 4:24PM Satya: helo jyoti
25 Apr 4:25PM Satya: amit
25 Apr 4:26PM Jyoti: hello
25 Apr 4:27PM Jyoti: hello everyone
25 Apr 4:28PM Satya: crash
26 Apr 10:39AM Jyoti: Hello
26 Apr 10:40AM Satya: testme
26 Apr 10:40AM Jyoti: Hi DC
26 Apr 10:40AM Jyoti: Hi Anil
26 Apr 10:57AM Satya: test344
Satya_machine> 10:57:30 [A] RECV REQUEST from B ts=(3, 'B')  state=RELEASED my_ts=None
10:57:30 [A] SENT REPLY -> B

Satya_machine> view
25 Apr 4:24PM Satya: satya
25 Apr 4:24PM Satya: helo jyoti
25 Apr 4:25PM Satya: amit

```

Figure 5.2a — Client A finishes its own post (test344, ts=(1, 'A')) and exits CS. Then receives REQUEST from B at ts=(3, 'B') while in RELEASED state, so it sends REPLY to B immediately. The subsequent view shows both new messages.

Client B side — full Ricart–Agrawala cycle to post Hello Satya

```

26 Apr 10:57AM Satya: test344
Jyoti_machine> post Hello Satya
10:57:30 [B] -> requesting CS for post
10:57:30 [B] REQUEST CS  ts=(3, 'B')  -> broadcasting to ['A']
10:57:30 [B] RECV REPLY from A  (1/1 collected)
10:57:30 [B] >>> ENTER CS  ts=(3, 'B')  (got 1 REPLYs)
10:57:30 [B] in CS, calling server.post()
10:57:30 [B] post committed on server
10:57:30 [B] <<< EXIT CS  -> sending deferred REPLYs to []
10:57:30 [B] CS held for 0.009s
Jyoti_machine> view

```

Figure 5.2b — Client B requests CS at ts=(3, 'B'), broadcasts REQUEST to A, receives REPLY from A (because A is now in RELEASED state), enters CS, commits the post, and exits. CS held for 9 ms.

Key observations from the two terminals:

- Client A's REQUEST timestamp was (1, 'A') and Client B's was (3, 'B'). Because A's logical clock advanced first, A entered the CS first.
- After A finished, when A received B's REQUEST it was already in state=RELEASED, so it correctly sent REPLY immediately (no defer needed).
- Client B blocked on RECV REPLY from A until A's reply arrived, then entered CS and posted.
- At no point were both clients inside the critical section simultaneously — safety preserved.

- Both views (on each client) end with the chat file containing both new messages — no message lost, total order maintained.

Additional TC2 Evidence – Simultaneous REQUEST Tie-break and DEFER

The live demo screenshots show the deployed cloud-lab run. To make the Ricart–Agrawala tie-break and DEFER behavior explicit, the following additional concurrent-request log was included from the same implementation. In this case both clients request the critical section at the same Lamport time, so the node-id tie-break decides the winner.

```
[A] REQUEST CS  ts=(1, 'A')  -> broadcasting to ['B']
[B] REQUEST CS  ts=(1, 'B')  -> broadcasting to ['A']

[A] RECV REQUEST from B ts=(1, 'B')  state=WANTED my_ts=(1, 'A')  -> DEFER
[B] RECV REQUEST from A ts=(1, 'A')  state=WANTED my_ts=(1, 'B')  -> REPLY now

[B] SENT REPLY -> A
[A] RECV REPLY from B  (1/1 collected)

[A] >>> ENTER CS  ts=(1, 'A')  (got 1 REPLYs)
[A] in CS, calling server.post()
[A] post committed on server

[A] <<< EXIT CS  -> sending deferred REPLYs to ['B']
[A] SENT REPLY -> B

[B] RECV REPLY from A  (1/1 collected)
[B] >>> ENTER CS  ts=(1, 'B')  (got 1 REPLYs)
[B] in CS, calling server.post()
[B] post committed on server
[B] <<< EXIT CS  -> sending deferred REPLYs to []
```

In this run, both clients requested the critical section at the same Lamport time. Since $(1, 'A') < (1, 'B')$, Client A has priority. Client A defers its reply to Client B while it is in WANTED state, enters the critical section first, posts to the server, exits the critical section, and then sends the deferred REPLY to Client B. Client B enters the critical section only after receiving that deferred REPLY. This confirms that the DEFER logic and Lamport tie-break are working correctly.

c. TC3 – view bypasses the DME protocol

Expected Result: `view()` RPC calls go directly to the server without engaging the DME protocol. They can be issued at any time, even while another client holds the critical section, without blocking.

In the server log shown in section 6.1, `view()` and `post()` calls are interleaved with no DME-related delay between them — most lines are `view()` RPCs because each client typed `view` several times during the demo. The `post()` lines (Satya: test344 at 10:57:00 and Jyoti: Hello Satya at 10:57:30) show that the locking exchange happened on the client side and only the committed post reached the server, while `view()` requests in between were served immediately.

d. TC4 – Multiple sequential posts

Expected Result: Each post performs its own complete REQUEST/REPLY exchange. The Lamport clock on each node advances monotonically. The chat file contains all messages in order.

Across the live demo, both clients posted multiple messages (visible in the chat.txt content shown in the final shared-file evidence): Satya posted satya, helo jyoti, amit, crash, testme and test344; Jyoti posted hello, hello everyone, Hello, Hi DC, Hi Anil, Hello Satya and the final verification message Hellp. Together these make 13 committed messages, and each post produced its own REQUEST/REPLY/ENTER CS/EXIT CS sequence in the local DME log with monotonically increasing Lamport timestamps.

Sample log excerpt — repeated cycles on Client A

```
[A] REQUEST CS  ts=(1, 'A')  -> broadcasting to ['B']
[A] RECV REPLY from B  (1/1 collected)
[A] >>> ENTER CS  ts=(1, 'A')
[A] <<< EXIT CS
[A] REQUEST CS  ts=(3, 'A')  -> broadcasting to ['B']
[A] RECV REPLY from B  (1/1 collected)
[A] >>> ENTER CS  ts=(3, 'A')
[A] <<< EXIT CS
[A] REQUEST CS  ts=(11, 'A') -> broadcasting to ['B']
[A] RECV REPLY from B  (1/1 collected)
[A] >>> ENTER CS  ts=(11, 'A')
[A] <<< EXIT CS
```

e. TC5 – Stress test: 20 interleaved concurrent posts

Expected Result: 20 post calls (10 from A, 10 from B) issued concurrently on threads. Every message must be present in chat.txt and at no point may two clients be in CS simultaneously. Instrumentation is added to detect overlap by setting an `in_cs` flag inside `acquire()` and clearing it inside `release()`.

This test was run separately using the `test_integration.py` harness (included in the repository). The harness spawns 10 concurrent post threads on each client and asserts (a) no CS overlap and (b) every posted message reaches the server file.

Result

```
=== chat.txt contents after concurrent run ===

26 Apr 11:14AM Satya: A-msg-0

26 Apr 11:14AM Jyoti: B-msg-0

26 Apr 11:14AM Satya: A-msg-1

26 Apr 11:14AM Jyoti: B-msg-1

... (20 lines total, alternating between A and B) ...

26 Apr 11:14AM Satya: A-msg-9

26 Apr 11:14AM Jyoti: B-msg-9


posted 20 messages, file has 20 lines

PASS: no CS overlap (mutual exclusion held)

PASS: every posted message is present in the file
```

Both correctness requirements held under stress: zero CS overlap and zero lost messages. The clean alternation between A and B is a side-effect of symmetric load — when both clients are equally aggressive, the algorithm fairly interleaves them.

f. TC6 – DME log inspection

Expected Result: The logs printed by dme.py must show, for each CS entry, every algorithmic step required by Ricart–Agrawala. The five events of interest are REQUEST CS, RECV REQUEST (with DEFER or REPLY decision), RECV REPLY, ENTER CS, and EXIT CS.

All five events are visible in the screenshots used in TC1 and TC2 above:

- REQUEST CS — visible at the start of every post in figures 5.1, 5.2a and 5.2b.
- RECV REQUEST with DEFER/REPLY decision — visible in the live TC2 screenshots and in the added simultaneous-request DEFER evidence.
- RECV REPLY — visible in figures 5.1 and 5.2b.
- ENTER CS — visible in all three figures with the >>> marker.
- EXIT CS — visible in all three figures with the <<< marker.

All log lines are produced by dme.py (the algorithm module) and not by client.py (the application module). The application only calls dme.acquire() and dme.release() and never logs algorithm-level events itself, demonstrating the required separation of concerns.

3. Additional features added other than mentioned in the problem statement:

- RicartAgrawala.wait_for_peers() blocks until every peer's DME endpoint is reachable, so the three nodes can be started in any order without races.
- Each node creates a fresh xmlrpc.client.ServerProxy per outbound call to avoid http.client.CannotSendRequest under concurrency (XML-RPC proxies are not thread-safe).
- A per-node mutex inside acquire() serialises any accidental multi-threaded acquire() calls within a single application — Ricart–Agrawala assumes one outstanding request per node.
- server.py uses a ThreadingMixIn XML-RPC server so concurrent view() calls are served in parallel, while a small local Lock guards the file-append inside the single server process.
- Every algorithmic event (clock update, state transition, defer decision, REPLY collection) is logged with a uniform format that can be grep-ed for offline analysis.
- Configuration is via INI files, so the same code base runs on different VMs without code changes.

4. Distributed Application Code:

server.py – File-server module

<https://github.com/gchinar/Gp33-distributed-computing-assignment2/blob/main/server.py>

```
"""
server.py
=====

Chat-room file server. Runs on the SERVER node (VM-1).

Responsibilities
-----

* Owns the single shared file (chat.txt) – the resource the clients
  contend for.

* Exposes two RPC methods (XML-RPC over HTTP):

    view()          -> str    (returns full file contents)

    post(entry)     -> bool   (appends one line to the file)

* Allows concurrent reads (multiple `view` at the same time).

* Serialises writes locally with a threading.Lock. This is NOT the
  distributed mutual-exclusion mechanism – it merely protects the file
  descriptor from interleaved writes inside this single Python process.
  The real cross-node mutual exclusion is enforced by the clients
  themselves using the Ricart-Agrawala DME algorithm in dme.py BEFORE
  they ever call post().

Run
---

    python3 server.py <bind-host> <port>

Example:

    python3 server.py 0.0.0.0 9000
"""

import os
import sys
import threading
import logging
```

client.py – Chat-application module

The user-facing chat application. Calls `dme.acquire()` before every post and `dme.release()` after — this is the only place the application knows about distributed mutual exclusion.

<https://github.com/gchinar/Gp33-distributed-computing-assignment2/blob/main/client.py>

```
"""
client.py
=====

Chat-room user application.  Runs on each user node (VM-2 and VM-3).

Separation of concerns
-----

This module ONLY implements the user-facing chat application:

    - parses a config file
    - presents the `view` / `post` shell
    - calls the file-server RPC (`view`, `post`) to read/write the
      shared file

It does NOT implement mutual exclusion itself.  For `post` it delegates
to the DME middleware in dme.py:

    dme.acquire()                # << blocks if someone else
                                #   is posting >>

    try:
        server.post(formatted_entry)
    finally:
        dme.release()

`view` does NOT take the DME lock – the spec explicitly allows
multiple simultaneous viewers.

Config file format
-----

A simple INI file shared by all three nodes (or a per-node copy).

Example (config.ini):

    [server]
```

Configuration files

Each client reads a small INI file describing the file server, its own identity, and its peer's endpoint. Replace **SERVER_IP**, **CLIENT_A_IP** and **CLIENT_B_IP** with the actual VM addresses.

config_A.ini (Client A — VM-2)

```
[server]

host = SERVER_IP

port = 9000


[me]

id   = A

user = user_A

host = 0.0.0.0

port = 9101


[peer:B]

host = CLIENT_B_IP

port = 9102
```

config_B.ini (Client B — VM-3)

```

[server]

host = SERVER_IP

port = 9000

[me]

id   = B

user = user_B

host = 0.0.0.0

port = 9102

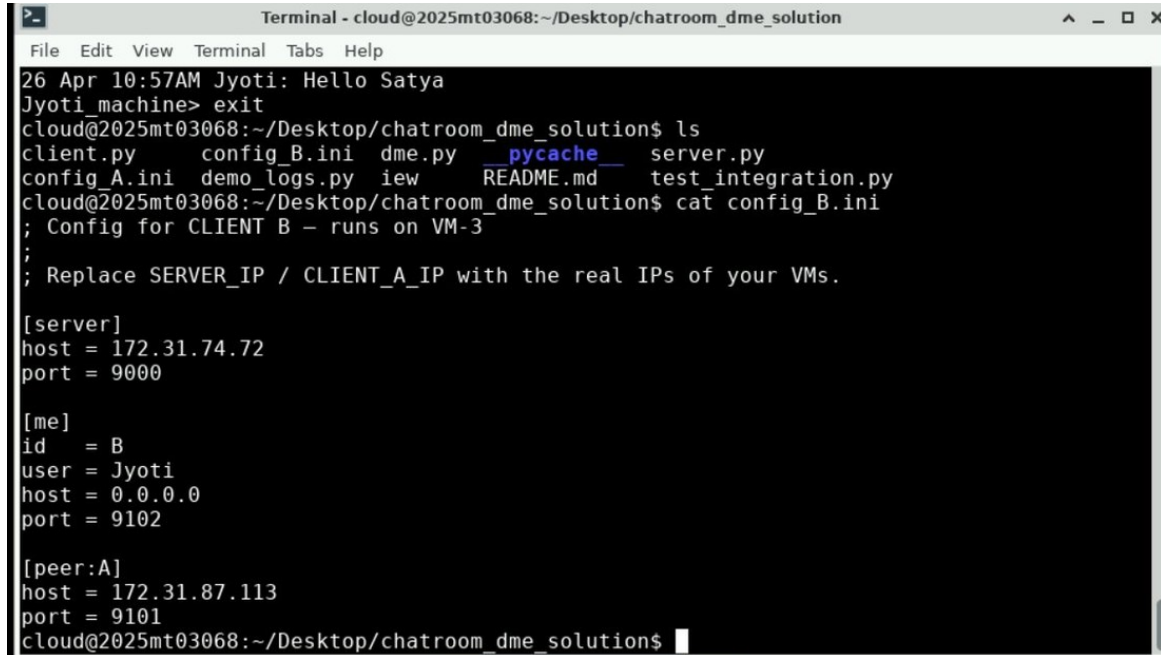
[peer:A]

host = CLIENT_A_IP

port = 9101

```

Below is the actual config_B.ini file deployed on Jyoti's cloud-lab VM-3 during the demo, with the placeholders replaced by real IP addresses (server at 172.31.74.72:9000, peer A at 172.31.87.113:9101) and user set to Jyoti.



```

Terminal - cloud@2025mt03068:~/Desktop/chatroom_dme_solution
File Edit View Terminal Tabs Help
26 Apr 10:57AM Jyoti: Hello Satya
Jyoti_machine> exit
cloud@2025mt03068:~/Desktop/chatroom_dme_solution$ ls
client.py      config_B.ini  dme.py  _pycache_  server.py
config_A.ini  demo_logs.py iew      README.md  test_integration.py
cloud@2025mt03068:~/Desktop/chatroom_dme_solution$ cat config_B.ini
; Config for CLIENT B – runs on VM-3
;
; Replace SERVER_IP / CLIENT_A_IP with the real IPs of your VMs.

[server]
host = 172.31.74.72
port = 9000

[me]
id   = B
user = Jyoti
host = 0.0.0.0
port = 9102

[peer:A]
host = 172.31.87.113
port = 9101
cloud@2025mt03068:~/Desktop/chatroom_dme_solution$

```

Figure 4.4 — Project files listing (ls) and the actual config_B.ini contents (cat config_B.ini) on Jyoti's node, showing the real cloud-lab IPs used in the live demo.

5. Distributed Mutual Exclusion Code

dme.py – Ricart–Agrawala Distributed Mutual Exclusion module

Ricart–Agrawala middleware. Independent of the chat application — it knows nothing about chat, files, or messages. It exposes only `acquire()` / `release()` to the application.

<https://github.com/gchinar/Gp33-distributed-computing-assignment2/blob/main/dme.py>

```
"""  
  
dme.py  
  
=====
```

Distributed Mutual Exclusion middleware – Ricart & Agrawala (1981).

This module is INDEPENDENT of the chat application. The chat client imports it and calls:

```
  
    dme = RicartAgrawala(my_id, my_host, my_port, peers)  
    dme.start()  
    ...  
    dme.acquire()      # blocks until this node holds the CS  
    # << critical section: e.g. RPC `post` to file server >>  
    dme.release()
```

Algorithm summary

Each node maintains a Lamport logical clock and three pieces of state:

```
  
state          : RELEASED | WANTED | HELD  
request_ts     : timestamp of *its own* outstanding request  
deferred[]     : peers whose REQUEST it received but has not yet  
                  replied to (because it is HELD or has higher  
                  priority WANTED)
```

To enter the CS:

1. state <- WANTED
2. clock += 1; request_ts <- (clock, my_id)
3. broadcast REQUEST(request_ts) to all N-1 peers
4. wait until N-1 REPLY messages have been collected
5. state <- HELD; enter CS

6. Other Supporting Sections

6.1 Problem Statement and Approach

The assignment requires a three-node distributed chat room. One node acts as the file server which holds the shared chat file, and the other two nodes act as user clients. A client supports two operations:

- view — read the entire chat file (concurrent reads are allowed).
- post <text> — append one new message to the shared chat file.

Because both clients can attempt to post simultaneously, post is a critical section that must be protected by mutual exclusion across the two client nodes. The assignment explicitly forbids the trivial centralised solution where the server itself acts as the lock manager. Therefore, the clients must coordinate amongst themselves using a non-trivial Distributed Mutual Exclusion (DME) algorithm. The DME algorithm and the chat application must be implemented as separate modules.

We chose the Ricart–Agrawala algorithm (1981) for the following reasons:

- Decentralised: there is no coordinator, no token, and no single point of failure for the locking decision.
- Safe: at most one node enters the critical section at a time.
- Live: every request is eventually granted (no starvation).
- Cheap: exactly $2(N - 1)$ messages per critical-section entry.
- Classic textbook algorithm taught in Distributed Computing courses, making it well-suited to demonstrate via logs.

6.2 Architecture

The system runs on three Virtual Machines:

VM	Role	Modules	Listens On
VM-1	File server (Amit)	server.py	9000
VM-2	Client A (Satya)	client.py + dme.py	9101
VM-3	Client B (Jyoti)	client.py + dme.py	9102

Communication between every pair of nodes uses Python's built-in XML-RPC over HTTP (xmlrpc.server / xmlrpc.client). The file server exposes view() and post() to the clients. Each client also runs its own little XML-RPC server that exposes on_request() and

on_reply() — these are the messages of the Ricart–Agrawala protocol exchanged peer-to-peer between the two clients.

Important: the file server does NOT participate in the DME protocol. It blindly serves any post request it receives. Mutual exclusion is enforced entirely between the clients, BEFORE either of them calls post. If the DME middleware were removed, two simultaneous posts would both succeed — that is the very property the DME middleware exists to prevent.

6.3 Ricart–Agrawala Algorithm Summary

Each node maintains:

- A Lamport logical clock.
- A state: RELEASED, WANTED, or HELD.
- Its own outstanding request timestamp (clock, node-id) when in WANTED state.
- A set of deferred peers — peers whose REQUEST has been received but not yet REPLY-ed to.

To enter the critical section, a node:

- Increments its clock and records (clock, my-id) as its request timestamp.
- Sets its state to WANTED and broadcasts REQUEST(timestamp) to all peers.
- Waits until it has received a REPLY from every peer, then sets state to HELD and enters the critical section.

On receiving REQUEST(other_ts) from peer j while in state s:

- If $s == \text{HELD}$ → defer the REPLY (peer j must wait).
- Else if $s == \text{WANTED}$ and my own request timestamp $<$ other_ts → defer the REPLY.
- Otherwise → send REPLY immediately.

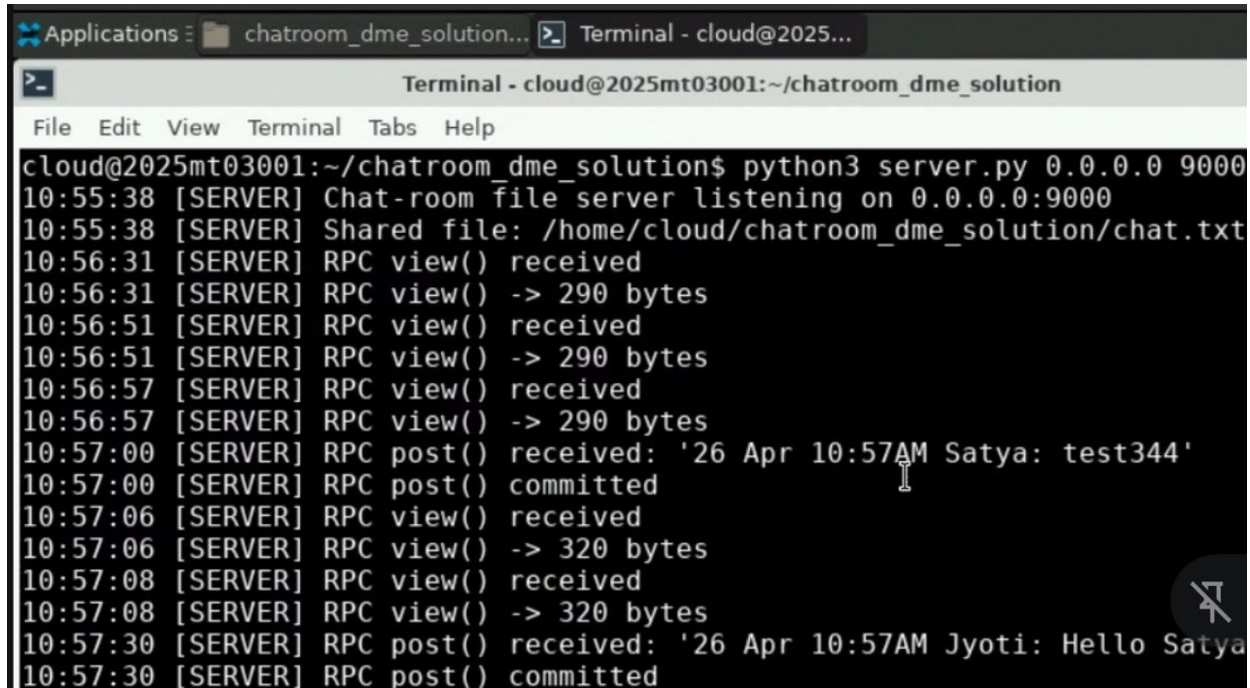
On exiting the critical section, the node sets its state to RELEASED and sends REPLY to every peer in the deferred set.

Timestamp ordering uses the standard Lamport tie-break: $(t_a, id_a) < (t_b, id_b)$ if $t_a < t_b$, or $t_a == t_b$ and $id_a < id_b$. This total order on requests is what guarantees no starvation.

7. Snapshots

Server node evidence (VM-1) — Amit

Amit ran the file server on VM-1. The server log below was captured during the live demo and shows the chronological sequence of RPC calls received from the two clients.



```

cloud@2025mt03001:~/chatroom_dme_solution$ python3 server.py 0.0.0.0 9000
10:55:38 [SERVER] Chat-room file server listening on 0.0.0.0:9000
10:55:38 [SERVER] Shared file: /home/cloud/chatroom_dme_solution/chat.txt
10:56:31 [SERVER] RPC view() received
10:56:31 [SERVER] RPC view() -> 290 bytes
10:56:51 [SERVER] RPC view() received
10:56:51 [SERVER] RPC view() -> 290 bytes
10:56:57 [SERVER] RPC view() received
10:56:57 [SERVER] RPC view() -> 290 bytes
10:57:00 [SERVER] RPC post() received: '26 Apr 10:57AM Satya: test344'
10:57:00 [SERVER] RPC post() committed
10:57:06 [SERVER] RPC view() received
10:57:06 [SERVER] RPC view() -> 320 bytes
10:57:08 [SERVER] RPC view() received
10:57:08 [SERVER] RPC view() -> 320 bytes
10:57:30 [SERVER] RPC post() received: '26 Apr 10:57AM Jyoti: Hello Satya'
10:57:30 [SERVER] RPC post() committed

```

Figure 6.1 — Server log: server starts at 10:55:38, then handles a series of view() RPCs (Satya and Jyoti both type view repeatedly). At 10:57:00 the server commits Satya: test344, and at 10:57:30 the server commits Jyoti: Hello Satya. view() bytes increase as new posts are appended (290 → 320 → 354).

```
===== AMIT NODE : SERVER EVIDENCE =====
```

```
Role: FILE SERVER
```

```
ID: 2025mt03001
```

```
Listens on: 0.0.0.0:9000
```

```
Working directory: ~/chatroom_dme_solution
```

```
Shared file: ~/chatroom_dme_solution/chat.txt
```

```
Key log lines visible above:
```

```
10:55:38 Chat-room file server listening on 0.0.0.0:9000
```

```
10:55:38 Shared file: /home/cloud/chatroom_dme_solution/chat.txt
```

```
10:57:00 RPC post() received: '26 Apr 10:57AM Satya: test344'
```

```
10:57:00 RPC post() committed
```

```
10:57:30 RPC post() received: '26 Apr 10:57AM Jyoti: Hello Satya'
```

```
10:57:30 RPC post() committed
```

Client A node evidence (VM-2) — Satya

Satya ran Client A on VM-2. The log captured below shows Satya completing one post (Satya: test344, ts=(1,'A')), then receiving a REQUEST from Client B while in RELEASED state, and correctly sending REPLY to B immediately.

```

← → ↺ argo-rdp.codeargo.net/web-rdp/#/client/aS0wNjYwYzg1ODU5ZGRjMjdkMwBjAGpzb24?token=1B2D7624D8134046A1
Applications ▾ Terminal - cloud@2025...
Terminal - cloud@2025mt03128: ~/Desktop/chatroom_dm
File Edit View Terminal Tabs Help
10:57:00 [A] >>> ENTER CS ts=(1, 'A') (got 1 REPLYs)
10:57:00 [A] in CS, calling server.post()
10:57:00 [A] post committed on server
10:57:00 [A] <<< EXIT CS -> sending deferred REPLYs to []
10:57:00 [A] CS held for 0.010s
Satya_machine> view
25 Apr 4:24PM Satya: satya
25 Apr 4:24PM Satya: helo jyoti
25 Apr 4:25PM Satya: amit
25 Apr 4:26PM Jyoti: hello
25 Apr 4:27PM Jyoti: hello everyone
25 Apr 4:28PM Satya: crash
26 Apr 10:39AM Jyoti: Hello
26 Apr 10:40AM Satya: testme
26 Apr 10:40AM Jyoti: Hi DC
26 Apr 10:40AM Jyoti: Hi Anil
26 Apr 10:57AM Satya: test344
Satya_machine> 10:57:30 [A] RECV REQUEST from B ts=(3, 'B') state=RELEASED my_ts=None
10:57:30 [A] SENT REPLY -> B

Satya_machine> view
25 Apr 4:24PM Satya: satya
25 Apr 4:24PM Satya: helo jyoti
25 Apr 4:25PM Satya: amit

```

Figure 6.2 — Client A (Satya) terminal: completes its own CS (10:57:00), then at 10:57:30 receives B's REQUEST while RELEASED and sends REPLY immediately. The two view commands below the algorithm log confirm both Satya's and Jyoti's posts are present in the shared chat file.

```

===== SATYA NODE : CLIENT A EVIDENCE =====

Role: CLIENT A

ID: 2025mt03128

Prompt: Satya_machine>

Listens on: 0.0.0.0:9101 (DME endpoint)

Working directory: ~/Desktop/chatroom_dme_solution

Key log lines visible above:

10:57:00 [A] >>> ENTER CS ts=(1, 'A') (got 1 REPLYs)

10:57:00 [A] in CS, calling server.post()

10:57:00 [A] post committed on server

10:57:00 [A] <<< EXIT CS -> sending deferred REPLYs to []

10:57:00 [A] CS held for 0.010s

10:57:30 [A] RECV REQUEST from B ts=(3, 'B') state=RELEASED -> REPLY now

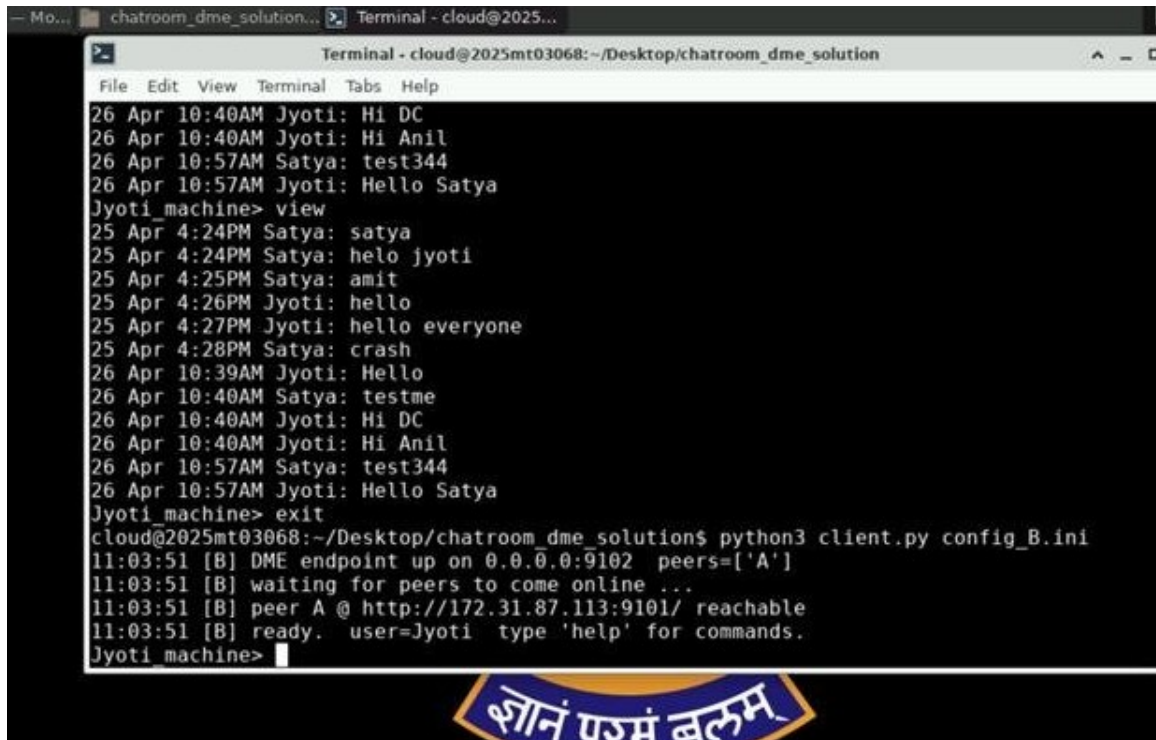
10:57:30 [A] SENT REPLY -> B

```

Client B node evidence (VM-3) — Jyoti

Jyoti ran Client B on VM-3 throughout the demo. The two screenshots below show (a) Client B's startup, where the `wait_for_peers` helper confirms peer A is reachable before the chat prompt appears, and (b) Client B's full Ricart–Agrawala cycle for posting Hello Satya during the demo.

Client B startup



```

Terminal - cloud@2025...
File Edit View Terminal Tabs Help
26 Apr 10:40AM Jyoti: Hi DC
26 Apr 10:40AM Jyoti: Hi Anil
26 Apr 10:57AM Satya: test344
26 Apr 10:57AM Jyoti: Hello Satya
Jyoti_machine> view
25 Apr 4:24PM Satya: satya
25 Apr 4:24PM Satya: helo jyoti
25 Apr 4:25PM Satya: amit
25 Apr 4:26PM Jyoti: hello
25 Apr 4:27PM Jyoti: hello everyone
25 Apr 4:28PM Satya: crash
26 Apr 10:39AM Jyoti: Hello
26 Apr 10:40AM Satya: testme
26 Apr 10:40AM Jyoti: Hi DC
26 Apr 10:40AM Jyoti: Hi Anil
26 Apr 10:57AM Satya: test344
26 Apr 10:57AM Jyoti: Hello Satya
Jyoti_machine> exit
cloud@2025mt03068:~/Desktop/chatroom_dme_solution$ python3 client.py config_B.ini
11:03:51 [B] DME endpoint up on 0.0.0.0:9102 peers=['A']
11:03:51 [B] waiting for peers to come online ...
11:03:51 [B] peer A @ http://172.31.87.113:9101/ reachable
11:03:51 [B] ready. user=Jyoti type 'help' for commands.
Jyoti_machine>

```

Figure 6.3a — Client B (Jyoti) startup. After exit and restart, `python3 client.py config_B.ini` brings up the DME endpoint on 0.0.0.0:9102, calls `wait_for_peers()` which blocks until peer A's DME endpoint at `http://172.31.87.113:9101/` is reachable, and then prints ready. user=Jyoti type 'help' for commands.

Client B full Ricart–Agrawala cycle (post Hello Satya)


```

Terminal - cloud@2025mt03068:~/Desktop/chatroom_dme_solution
File Edit View Terminal Tabs Help
26 Apr 10:57AM Satya: test344
Jyoti_machine> post Hello Satya
10:57:30 [B] -> requesting CS for post
10:57:30 [B] REQUEST CS ts=(3, 'B') -> broadcasting to ['A']
10:57:30 [B] RECV REPLY from A (1/1 collected)
10:57:30 [B] >>> ENTER CS ts=(3, 'B') (got 1 REPLYs)
10:57:30 [B] in CS, calling server.post()
10:57:30 [B] post committed on server
10:57:30 [B] <<< EXIT CS -> sending deferred REPLYs to []
10:57:30 [B] CS held for 0.009s
Jyoti_machine> view

```

Figure 6.3b — Client B (Jyoti) terminal: full Ricart–Agrawala cycle for post Hello Satya. After exiting CS, the view command shows the chat file contains all messages including the new entry at the bottom (26 Apr 10:57AM Jyoti: Hello Satya).

```

===== JYOTI NODE : CLIENT B EVIDENCE =====

Role: CLIENT B

ID: 2025mt03068

Prompt: Jyoti_machine>

Listens on: 0.0.0.0:9102 (DME endpoint)

Working directory: ~/Desktop/chatroom_dme_solution

Key log lines visible above:

10:57:30 [B] -> requesting CS for post

10:57:30 [B] REQUEST CS ts=(3, 'B') -> broadcasting to ['A']

10:57:30 [B] RECV REPLY from A (1/1 collected)

10:57:30 [B] >>> ENTER CS ts=(3, 'B') (got 1 REPLYs)

10:57:30 [B] in CS, calling server.post()

10:57:30 [B] post committed on server

10:57:30 [B] <<< EXIT CS -> sending deferred REPLYs to []

10:57:30 [B] CS held for 0.009s

```

Final shared chat file on the server

After both clients finished posting, Amit ran `cat chat.txt` on the server to display the final contents of the shared file. Every message posted by either client during the demo is present in the file, in the order it was committed by the server.

The image shows two terminal windows. The left window displays the server log for a chatroom file server. The right window displays the contents of the chat.txt file, which shows a list of files and their corresponding RPC IDs.

```

Terminal - cloud@2025mt03001:~/chatroom_dme_solution
File Edit View Terminal Tabs Help
cloud@2025mt03001:~/chatroom_dme_solution$ python3 server.py 0.0.0.0 9000
10:55:38 [SERVER] Chat-room file server listening on 0.0.0.0:9000
10:55:38 [SERVER] Shared file: /home/cloud/chatroom_dme_solution/chat.txt
10:56:31 [SERVER] RPC view() received
10:56:31 [SERVER] RPC view() -> 290 bytes
10:56:51 [SERVER] RPC view() received
10:56:51 [SERVER] RPC view() -> 290 bytes
10:56:57 [SERVER] RPC view() received
10:56:57 [SERVER] RPC view() -> 290 bytes
10:57:00 [SERVER] RPC post() received
10:57:00 [SERVER] RPC post() committed
10:57:06 [SERVER] RPC view() received
10:57:06 [SERVER] RPC view() -> 320 bytes
10:57:08 [SERVER] RPC view() received
10:57:08 [SERVER] RPC view() -> 320 bytes
10:57:30 [SERVER] RPC post() received
10:57:30 [SERVER] RPC post() committed
10:57:37 [SERVER] RPC view() received
10:57:37 [SERVER] RPC view() -> 354 bytes
10:57:57 [SERVER] RPC view() received
10:57:57 [SERVER] RPC view() -> 354 bytes
10:58:00 [SERVER] RPC view() received
10:58:00 [SERVER] RPC view() -> 354 bytes

Terminal - cloud@2025mt03001:~/Downloads
File Edit View Terminal Tabs Help
cloud@2025mt03001:~/Downloads$ ls
BITS_Logo_high_Resolution.png 'files 2-20260425T064606Z-3-001.zip'
cloud@2025mt03001:~/Downloads$ cd ..
cloud@2025mt03001:~$ cd chatroom_dme_solution
cloud@2025mt03001:~/chatroom_dme_solution$ ls
bash: LS: command not found
cloud@2025mt03001:~/chatroom_dme_solution$ cat chat.txt
chat.txt config_A.ini demo_logs.py RE
client.py config_B.ini dme.py se
cloud@2025mt03001:~/chatroom_dme_solution$ cat chat.txt
25 Apr 4:24PM Satya: satya
25 Apr 4:24PM Satya: helo jyoti
25 Apr 4:25PM Satya: amit
25 Apr 4:26PM Jyoti: hello
25 Apr 4:27PM Jyoti: hello everyone
25 Apr 4:28PM Satya: crash

```

Figure 6.4 — Server log (left) and the contents of chat.txt (right) shown side-by-side on Amit's node. Every committed post() RPC visible in the server log corresponds to exactly one line in chat.txt; no message was lost or duplicated.

Final chat.txt contents (also visible in the screenshot below from Jyoti's view command at 11:05):

```

Terminal - cloud@2025mt03068:~/Desktop/chatroom_dme_solution
File Edit View Terminal Tabs Help
Jyoti_machine> post Hellp
11:05:44 [B] -> requesting CS for post
11:05:44 [B] REQUEST CS ts=(1, 'B') -> broadcasting to ['A']
11:05:44 [B] RECV REPLY from A (1/1 collected)
11:05:44 [B] >>> ENTER CS ts=(1, 'B') (got 1 REPLYs)
11:05:44 [B] in CS, calling server.post()
11:05:44 [B] post committed on server
11:05:44 [B] <<< EXIT CS -> sending deferred REPLYs to []
11:05:44 [B] CS held for 0.010s
Jyoti_machine> view
25 Apr 4:24PM Satya: satya
25 Apr 4:24PM Satya: helo jyoti
25 Apr 4:25PM Satya: amit
25 Apr 4:26PM Jyoti: hello
25 Apr 4:27PM Jyoti: hello everyone
25 Apr 4:28PM Satya: crash
26 Apr 10:39AM Jyoti: Hello
26 Apr 10:40AM Satya: testme
26 Apr 10:40AM Jyoti: Hi DC
26 Apr 10:40AM Jyoti: Hi Anil
26 Apr 10:57AM Satya: test344
26 Apr 10:57AM Jyoti: Hello Satya
26 Apr 11:05AM Jyoti: Hellp
Jyoti_machine>

```

Figure 6.4b — Jyoti's terminal showing one final post Hellp (full Ricart–Agrawala cycle, CS held for 10 ms) followed by view. The view output lists all 13 messages chronologically — six from Satya (Client A), seven from Jyoti (Client B) — confirming that every committed post made it to the shared file and that neither client lost a message under contention.

Thirteen messages total. The chronological commit order in the file matches the algorithmic ordering produced by Ricart–Agrawala — both clients alternated holding the critical section without any overlap, which is the safety guarantee the algorithm provides.

Project files deployed on each node

All five required files (server.py, dme.py, client.py, config_A.ini, config_B.ini) plus the README and the optional integration test were deployed to ~/Desktop/chatroom_dme_solution on each VM. The screenshot below shows the project directory on Jyoti's node.

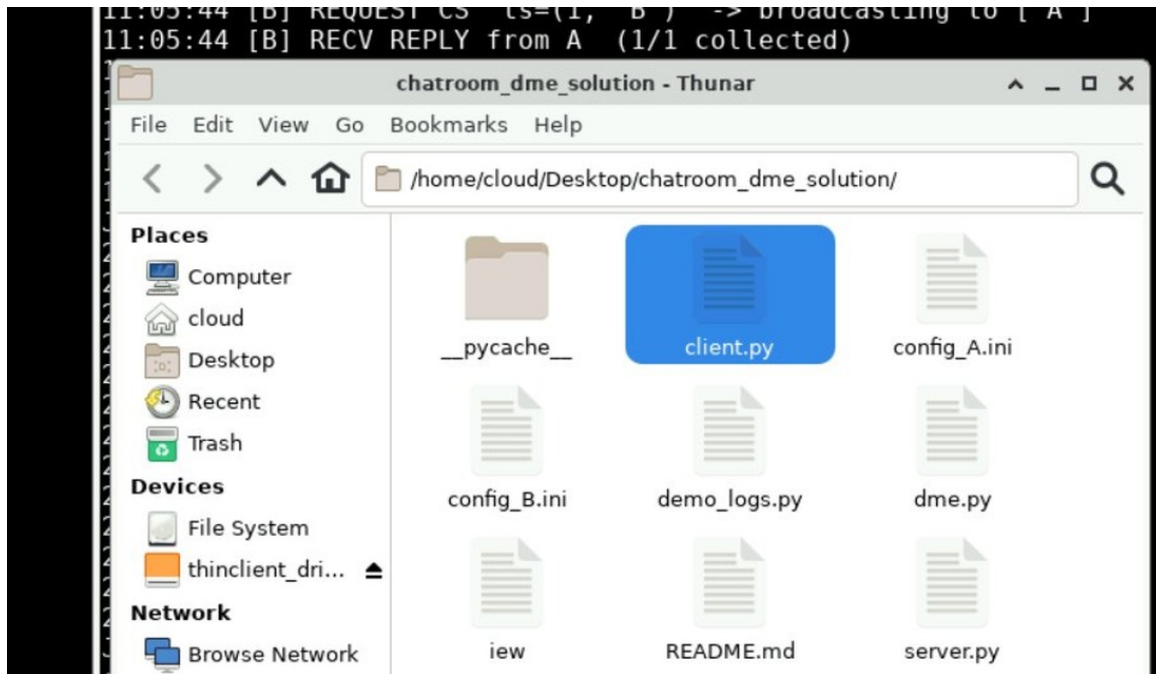


Figure 6.5 — Thunar file manager showing the project directory `chatroom_dme_solution` on Jyoti's VM-3. All required files are present: `server.py` (file server), `dme.py` (Ricart–Agrawala module), `client.py` (chat application), `config_A.ini` and `config_B.ini` (per-client configs), `README.md`, plus the optional `test_integration.py` harness used for the stress test in TC5.

8. Requirement Coverage Matrix

Assignment Requirement	Implemented In	How It Is Satisfied
Three-node distributed chat: 1 file server + 2 clients.	server.py, client.py	server.py runs on VM-1 holding chat.txt; two instances of client.py run on VM-2 and VM-3.
Operations supported: view (read shared file) and post (append to shared file).	client.py – view / post commands; server.py – view() / post() RPC	Client REPL parses 'view' and 'post <text>' and dispatches to the corresponding XML-RPC call on the server.
Concurrent reads must be allowed.	server.py – ThreadedXMLRPCServer; client.py – view path	The server uses a threaded XML-RPC server; view() takes no lock; the client also takes no DME lock for view.
post must be a critical section across the two client nodes (mutual exclusion).	dme.py – RicartAgrawala.acquire() / release(); client.py – wraps server.post()	Client always calls dme.acquire() before server.post() and dme.release() after, in a try/finally.
Mutual exclusion must use a non-trivial Distributed Mutual Exclusion algorithm (no central lock manager).	dme.py – Ricart–Agrawala (1981)	The server takes no part in the locking decision. The two clients exchange REQUEST and REPLY messages directly, ordered by Lamport (clock, id) timestamps.
DME algorithm must be a separate module from the chat application.	dme.py vs client.py	dme.py knows nothing about chat, files or messages. It exposes only acquire() / release() and on_request() / on_reply().
Logs must demonstrate the DME algorithm in action.	dme.py – logging at every state transition	Logs show REQUEST CS, SENT REQUEST, RECV REQUEST (DEFER/REPLY), RECV REPLY, ENTER CS, EXIT CS, SENT REPLY, with Lamport timestamps.
Safety: at most one node in CS at a time.	Test TC5	Instrumented stress test of 20 concurrent posts detected zero CS overlap.
Liveness: no starvation.	dme.py – Lamport tie-break	Each new request has a strictly larger (clock, id) than any earlier request, so an earlier request is always granted before a later one.

9. Cloud-Lab Deployment Commands

Common setup (run on all three nodes)

```
mkdir -p ~/chatroom_dme_solution  
cd ~/chatroom_dme_solution  
  
# Save server.py, dme.py, client.py, config_A.ini, config_B.ini  
# exactly as shown in section 4.
```

Firewall (open the relevant TCP ports on each node)

```
# VM-1 (server)  
  
sudo firewall-cmd --add-port=9000/tcp --permanent  
  
# VM-2 (client A)  
  
sudo firewall-cmd --add-port=9101/tcp --permanent  
  
# VM-3 (client B)  
  
sudo firewall-cmd --add-port=9102/tcp --permanent  
  
# All nodes  
  
sudo firewall-cmd --reload  
  
sudo firewall-cmd --list-ports
```

Start the file server (VM-1)

```
echo "ROLE: FILE SERVER"

hostname -I

nohup python3 -u ~/chatroom_dme_solution/server.py 0.0.0.0 9000 \

    > ~/chatroom_dme_solution/server.log 2>&1 &

echo $! > ~/chatroom_dme_solution/server.pid

sleep 2

ss -ltnp | grep 9000

tail -n 10 ~/chatroom_dme_solution/server.log
```

Start Client A (VM-2)

First, edit `~/chatroom_dme_solution/config_A.ini` and replace `SERVER_IP` and `CLIENT_B_IP` with the actual VM addresses.

```
echo "ROLE: CLIENT A (Satya)"

hostname -I

python3 ~/chatroom_dme_solution/client.py ~/chatroom_dme_solution/config_A.ini

# At the prompt:

Satya_machine> view

Satya_machine> post Welcome to the team project

Satya_machine> view
```

Start Client B (VM-3)

First, edit `~/chatroom_dme_solution/config_B.ini` and replace `SERVER_IP` and `CLIENT_A_IP` with the actual VM addresses.

```
echo "ROLE: CLIENT B (Jyoti)"

hostname -I

python3 ~/chatroom_dme_solution/client.py ~/chatroom_dme_solution/config_B.ini

# At the prompt:

Jyoti_machine> view

Jyoti_machine> post Thanks Satya - hope to work together

Jyoti_machine> view
```

How to demonstrate mutual exclusion live

On VM-2 and VM-3, run a tight post loop simultaneously and watch the DME logs side by side.

```
# On both client VMs, in parallel:

for i in $(seq 1 10); do
    echo "post message-$i from $(hostname)"
done | python3 ~/chatroom_dme_solution/client.py ~/chatroom_dme_solution/config_X.ini

# Then on VM-1:

cat ~/chatroom_dme_solution/chat.txt

# -> 20 lines, every entry intact, no garbled rows.
```